



Week 04 Task-12

OWASP Top 10 Web Vulnerabilities Overview

CandidateName: Asad Ali

Reference Number: TBS-INT-2026-445

Submission Date: 12|9|2026

Submitted To: TechBiz Security

Table of Contents

Executive Summary	3
Scope of Task	3
OWASP Top 10 Overview	3
A01:Broken Access Control	5
A02:Cryptographic Failures	5
A03:Injection	5
TryHackme Room Broken Access Control	6
TryHackme Room IAAA Failures	6
TryHackMe Room Insecure Data Handling	7
TryHackMe Room Application Design flaws	7
Conclusion	8

Executive Summary

This task focused on developing an understanding of the **OWASP Top 10 (2021)**, which identifies the most critical security risks affecting web applications. The task covered the meaning and security impact of each OWASP vulnerability, with additional study of Broken Access Control, Cryptographic Failures, and Injection. Real-world examples were reviewed to understand how these vulnerabilities can affect organizations.

Scope of Task

The scope of this task included studying the OWASP Top 10 (2021) web application security risks and understanding their potential impact. It covered one-line descriptions of all ten vulnerabilities, detailed study of the top three vulnerabilities, and review of real-world breach examples. The task also included completing the assigned TryHackMe OWASP Top 10 learning activity.

OWASP Top 10 Overview

The OWASP Top 10 is a widely recognized list of the most critical web application security risks. It is developed by the **Open Worldwide Application Security Project (OWASP)** to raise awareness of common security weaknesses. The list helps developers and security professionals understand, identify, and reduce vulnerabilities in web applications. This task provides an overview of the OWASP Top 10 (2021) vulnerabilities and their potential security impact.

A01: Broken Access Control: Improper access control restrictions can allow users to access or perform actions that they are not authorized to perform.

Example: In 2019, Facebook disclosed a vulnerability that allowed attackers to access information belonging to other users through improper authorization controls. This demonstrates how weaknesses in access control can allow unauthorized users to access data or functionality they should not have access to.

A02: Cryptographic Failures: Weak or missing encryption can expose sensitive data such as passwords, personal information, or financial details.

Example: The Yahoo data breaches demonstrated the risks associated with inadequate protection of user credentials and sensitive information. Weak password protection and cryptographic practices can make stolen credentials significantly easier for attackers to compromise.

A03: Injection: Untrusted user input can be interpreted as commands or queries, potentially allowing attackers to manipulate the application or its data.

Example: In 2015, UK telecommunications company TalkTalk suffered a major data breach after attackers exploited SQL Injection vulnerabilities in its websites. The

attack allowed access to customer information, affecting approximately 156,959 customers.

A04: Insecure Design: Security weaknesses can result from poor application design and the absence of appropriate security controls.

Example: The 2019 Capital One incident can also be viewed from an insecure-design perspective because security controls around access to cloud resources were not sufficiently designed to prevent unauthorized access. The incident demonstrates how security weaknesses in the design of application and cloud architecture can lead to significant data exposure.

A05: Security Misconfiguration: Incorrect or insecure configuration of applications, servers, or security settings can expose systems to attacks.

Example: In 2019, Capital One experienced a data breach involving a configuration vulnerability in its cloud infrastructure. Capital One stated that the attacker exploited a configuration weakness, which allowed unauthorized access to certain customer information.

A06: Vulnerable and Outdated Components: Using outdated or vulnerable software components can expose an application to known security vulnerabilities.

Example: In 2017, Equifax suffered a major breach after attackers exploited a known vulnerability in the Apache Struts framework used by its web application. The vulnerability had been disclosed and a patch was available, but the affected system remained vulnerable, resulting in the exposure of personal information of approximately 143 million U.S. consumers.

A07: Identification and Authentication Failures: Weak authentication or session management can allow attackers to compromise user accounts or impersonate legitimate users.

Example: In 2023, attackers used credential-stuffing techniques against 23andMe accounts by using previously compromised usernames and passwords. The incident demonstrates how weak authentication protections and password reuse can allow attackers to gain unauthorized access to user accounts.

A08: Software and Data Integrity Failures: Inadequate protection of software updates, code, or data can allow unauthorized modification or malicious components to be introduced.

Example: The 2020 SolarWinds **SUNBURST** attack is a major example of a software integrity failure. Attackers compromised the SolarWinds Orion build environment and inserted malicious code into legitimate software updates, causing customers to unknowingly receive compromised updates.

A09: Security Logging and Monitoring Failures: Insufficient logging and monitoring can prevent organizations from detecting, investigating, or responding to security incidents.

Example: The 2015 TalkTalk breach also demonstrated weaknesses in security monitoring. The ICO investigation found that previous SQL Injection attacks had occurred before the major breach, but appropriate monitoring was not in place to identify the vulnerabilities and attacks earlier.

A10: Server-Side Request Forgery (SSRF): An application can be manipulated into making unauthorized requests to internal or external resources on behalf of an attacker.

Example: In 2021, attackers exploited an SSRF vulnerability in Microsoft Exchange Server that allowed them to send unauthorized requests from the Exchange server to internal resources. The vulnerability was part of the ProxyLogon attack chain and was widely exploited against vulnerable Exchange servers.

A01:Broken Access Control

Broken Access Control occurs when an application does not properly enforce restrictions on what users are allowed to access or perform. This can allow users to access other users' data, modify resources, or perform actions without proper authorization. For example, changing an account ID in a URL could potentially allow a user to view another user's information.

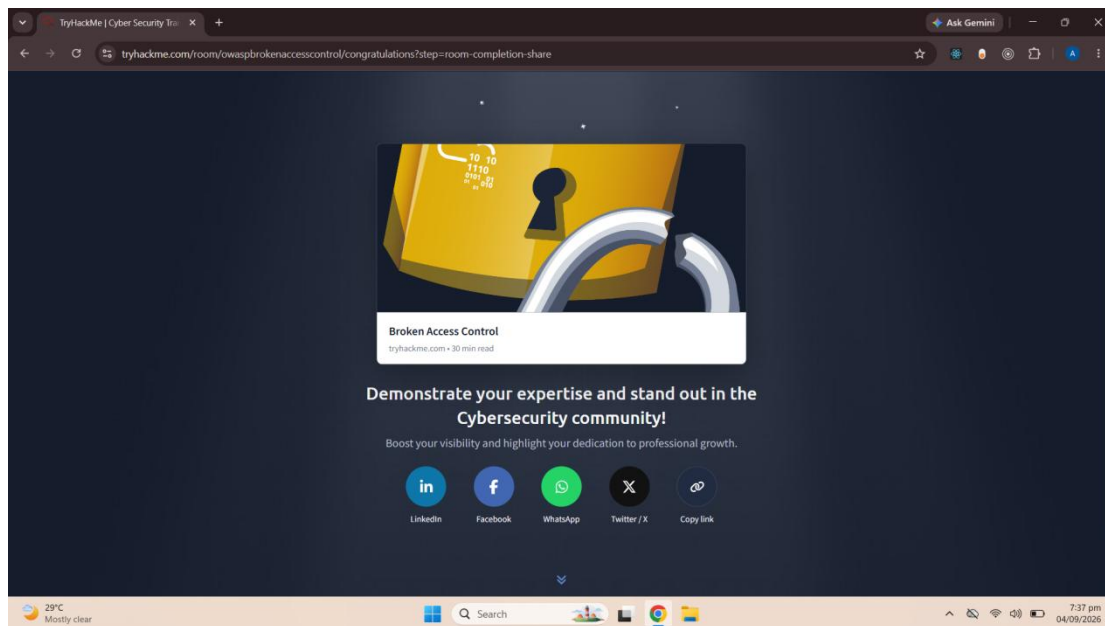
A02:Cryptographic Failures

Cryptographic Failures occur when sensitive information is not properly protected through appropriate encryption and security practices. This can expose passwords, personal information, financial data, or other confidential information. For example, transmitting sensitive data without HTTPS can allow the information to be exposed during communication.

A03:Injection

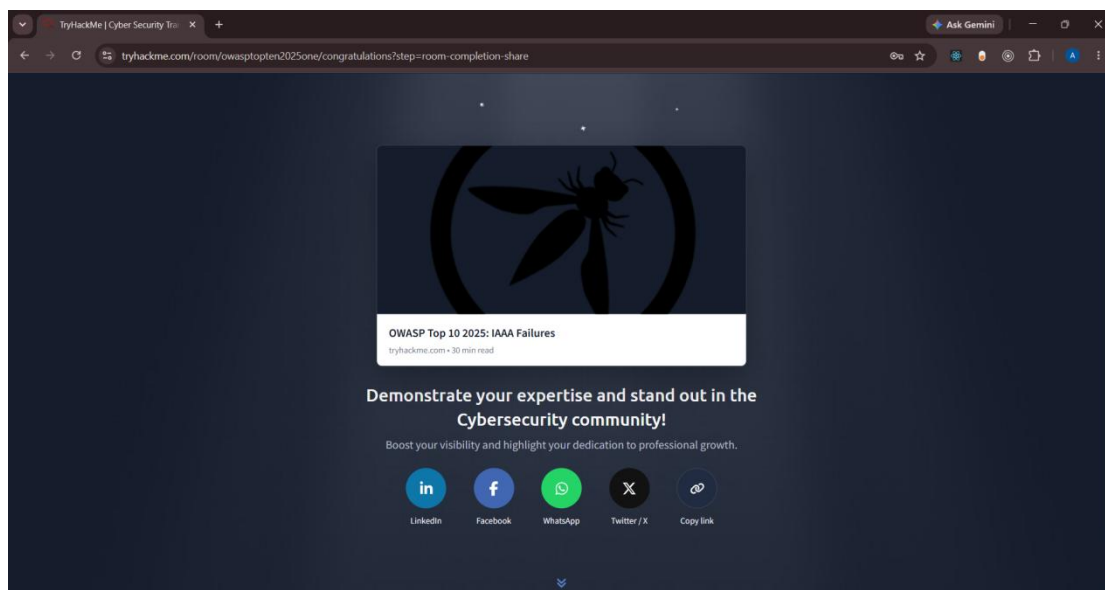
Injection occurs when untrusted user input is improperly processed by an application and is interpreted as commands, queries, or code. Injection is a broad category that includes different types such as SQL Injection, NoSQL Injection, LDAP Injection, ORM Injection, and Command Injection, while Cross-Site Scripting (XSS) is also considered an injection-related attack involving the injection of malicious scripts into web pages. These attacks can allow an attacker to manipulate databases, execute commands, or affect how an application processes and displays user-controlled data.

TryHackme Room Broken Access Control



The above TryHackMe **Broken Access Control** room provided practical understanding of how improper authorization controls can allow users to access resources or perform actions they are not authorized to perform. The exercises helped me understand common access control weaknesses and their potential impact on web application security.

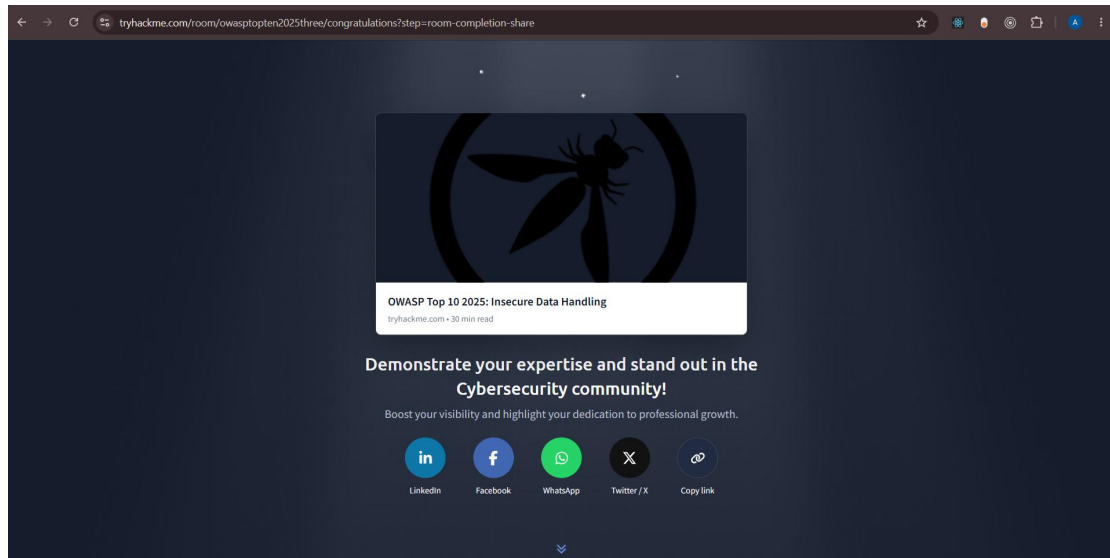
TryHackme Room IAAA Failures



The above TryHackMe IAAA Failures (2025) room provided practical exposure to **Broken Access Control (A01)**, **Identification and Authentication Failures (A07)**, and **Security Logging and Monitoring Failures (A09)**. The practical exercises

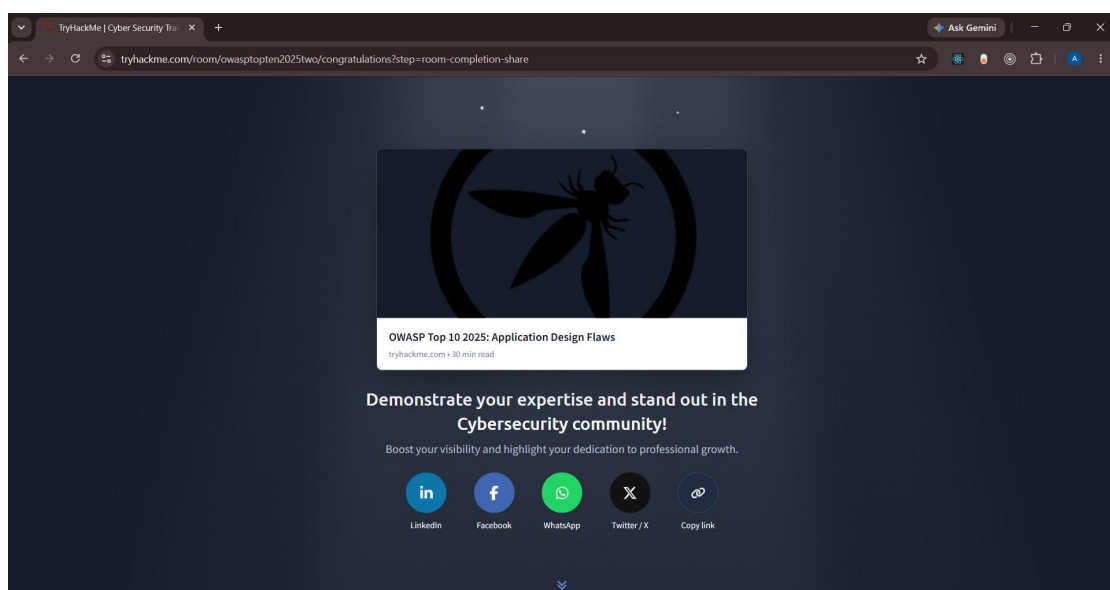
helped me understand how weaknesses in authorization, authentication, and security monitoring can affect the security of web applications.

TryHackMe Room Insecure Data Handling

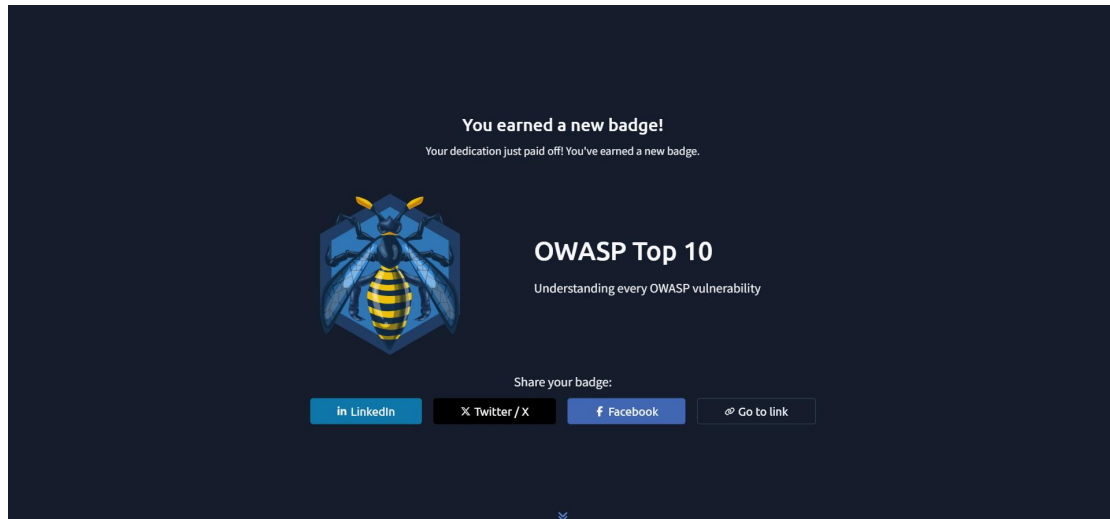


The above TryHackMe Insecure Data Handling room provided practical exposure to **Injection**, **Server Side Injection Template (SSTI)** and **Cryptographic Failures** through hands-on security exercises. The room helped me understand how improper input handling and weak protection of sensitive data can lead to security vulnerabilities in web applications.

TryHackMe Room Application Design flaws



The above TryHackMe **Application Design Flaws** room provided practical exposure to web application security issues related to **Cryptographic Failures (A02)**, **Injection (A03)**, **Vulnerable and Outdated Components (A06)**, and **Server-Side Request Forgery (A10)**. The exercises helped me understand how design and implementation weaknesses can be identified and how they may affect the security of web applications.



After completing the assigned OWASP vulnerability learning activities, I earned the **OWASP Top 10 badge**, as shown in the figure above. This achievement demonstrates my understanding of the OWASP Top 10 vulnerabilities and their potential impact on web application security.

Conclusion

This task provided a practical understanding of the OWASP Top 10 (2021) and the major security risks commonly found in web applications. The study covered all ten vulnerabilities, with a detailed focus on **Broken Access Control**, **Cryptographic Failures**, and **Injection**, including different forms of injection attacks. As a practical activity, the TryHackMe **OWASP Top 10 room** was also completed to gain hands-on experience. Overall, the task improved my understanding of common web vulnerabilities and the importance of applying appropriate security controls during application development and security testing.